

```

/**
 * =====
 * ANALYSTIQ - FULL BACKEND (CLEANED + PRICING TIERS)
 * =====
 * Tiers:
 *   FREE    - short-term chat memory only, no live feeds
 *   CORE    - $14.99/mo or $119/yr (recurring subscription)
 *   MASTER  - $299 one-time payment: lifetime access + the full PDF guide
 *
 * SECURITY: every API key, Stripe secret, and Stripe price ID is read from
 * an environment variable below - NONE of them are hardcoded in this file.
 * Set the real values in your hosting provider's environment settings
 * (Render/Railway/Vercel dashboard → Environment Variables), never in code.
 * If this file is ever committed to a public repo or pasted somewhere,
 * there is nothing secret sitting inside it to rotate.
 * =====
 */

const express = require('express');
const cors = require('cors');
const { PrismaClient } = require('@prisma/client');
const WebSocket = require('ws');
const fetch = require('node-fetch');
const { Anthropic } = require('@anthropic-ai/sdk');
const Stripe = require('stripe');

const app = express();
const prisma = new PrismaClient();

// =====
// ENV / CONFIG VAULT - reads only. Fill in actual values as environment
// variables on your host, not here.
// =====
const ENV = {
  DATABASE_URL: process.env.DATABASE_URL, // 🔒 set on hc
  FINNHUB_STOCK_API_KEY: process.env.FINNHUB_STOCK_API_KEY, // 🔒 set on h
  ANTHROPIC_API_KEY: process.env.ANTHROPIC_API_KEY, // 🔒 set on h
  STRIPE_SECRET_KEY: process.env.STRIPE_SECRET_KEY, // 🔒 set on h
  STRIPE_WEBHOOK_SECRET: process.env.STRIPE_WEBHOOK_SECRET, // 🔒 set on h
  JWT_SECRET: process.env.JWT_SECRET, // 🔒 set on h

  // CONFIGURE: paste each Stripe Price ID (from Dashboard → Product → Pricing)
  // into your host's environment variables under these exact names.
  STRIPE_PRICE_CORE_MONTHLY: process.env.STRIPE_PRICE_CORE_MONTHLY, // 🔒 $14.99/
  STRIPE_PRICE_CORE_ANNUAL: process.env.STRIPE_PRICE_CORE_ANNUAL, // 🔒 $119/yr

```

```

    STRIPE_PRICE_MASTER_KEY: process.env.STRIPE_PRICE_MASTER_KEY,      // 🔒 $299 on
    STRIPE_PRICE_EBOOK: process.env.STRIPE_PRICE_EBOOK,                // 🔒 $50 ebc
  };

// Fail loudly at startup if something critical is missing, instead of
// silently running with broken payments or AI calls.
function warnIfMissing(name, value) {
  if (!value) console.warn(`! Missing environment variable: ${name}`);
}
Object.entries(ENV).forEach(([key, value]) => warnIfMissing(key, value));

const anthropic = new Anthropic({ apiKey: ENV.ANTHROPIC_API_KEY });
const stripe = Stripe(ENV.STRIPE_SECRET_KEY);
const bcrypt = require('bcryptjs');
const jwt = require('jsonwebtoken');

app.use(cors());

// =====
// PRISMA SCHEMA (prisma/schema.prisma) - reference only, not executed here.
// Save this block into prisma/schema.prisma and run `npx prisma migrate dev`.
// =====
/*
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

enum SubscriptionTier {
  FREE
  CORE
  MASTER
}

model User {
  id          String    @id @default(uuid())
  email       String    @unique
  passwordHash String
  tier         SubscriptionTier @default(FREE)
  stripeCustomerId String? @unique
  stripeSubscriptionId String? @unique
  masterKeyPurchased Boolean @default(false)
  chatMessages ChatMessage[]

```

```

    memoryProfile      UserMemoryProfile?
    auditLogs           AuditLog[]
    createdAt           DateTime @default(now())
    updatedAt           DateTime @updatedAt
}

model UserMemoryProfile {
  id      String @id @default(uuid())
  userId  String @unique
  user    User    @relation(fields: [userId], references: [id], onDelete: Casc
  summary String @db.Text
  updatedAt DateTime @updatedAt
}

model ChatMessage {
  id      String @id @default(uuid())
  userId  String
  user    User    @relation(fields: [userId], references: [id], onDelete: Casc
  role    String
  content String @db.Text
  createdAt DateTime @default(now())
  @@index([userId, createdAt])
}

// Referenced by the research handler below – was missing from the
// original schema, added here so that handler doesn't crash at runtime.
model FinancialKnowledge {
  id      String @id @default(uuid())
  topic   String @unique
  content String @db.Text
}

model AuditLog {
  id      String @id @default(uuid())
  userId  String?
  user    User?   @relation(fields: [userId], references: [id], onDelete: SetN
  prompt  String @db.Text
  response String @db.Text
  status  String
  createdAt DateTime @default(now())
}

*/

// =====
// A0. AUTH – signup / login / sessions
// =====
function signToken(user) {

```

```

    return jwt.sign(
      { userId: user.id, email: user.email },
      ENV.JWT_SECRET,
      { expiresIn: '30d' }
    );
  }

// Verifies the token on requests that need to know who's asking. Attaches
// req.authUserId so route handlers use the verified identity instead of
// trusting a plain userId sent in the request body.
function requireAuth(req, res, next) {
  const header = req.headers['authorization'];
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ error: "Not logged in." });
  }
  const token = header.slice(7);
  try {
    const payload = jwt.verify(token, ENV.JWT_SECRET);
    req.authUserId = payload.userId;
    next();
  } catch (err) {
    return res.status(401).json({ error: "Your session has expired. Please log in" });
  }
}

app.post('/api/auth/signup', express.json(), async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ error: "Email and password are both required." });
    }
    if (password.length < 8) {
      return res.status(400).json({ error: "Password must be at least 8 character" });
    }

    const existing = await prisma.user.findUnique({ where: { email: email.toLowerCase() } });
    if (existing) {
      return res.status(409).json({ error: "An account with that email already exists" });
    }

    const passwordHash = await bcrypt.hash(password, 12);
    const user = await prisma.user.create({
      data: { email: email.toLowerCase(), passwordHash, tier: 'FREE' }
    });

    const token = signToken(user);
    res.status(201).json({ token, user: { id: user.id, email: user.email, tier: user.tier } });
  }
});

```

```

    } catch (error) {
      console.error("Signup error:", error);
      res.status(500).json({ error: "Could not create account. Please try again." })
    }
  });

```

```

app.post('/api/auth/login', express.json(), async (req, res) => {
  try {
    const { email, password } = req.body;
    if (!email || !password) {
      return res.status(400).json({ error: "Email and password are both required."
    }

    const user = await prisma.user.findUnique({ where: { email: email.toLowerCase
    // Same generic error whether the email doesn't exist or the password is
    // wrong - this is deliberate, so a login attempt can't be used to check
    // which emails have accounts.
    if (!user) return res.status(401).json({ error: "Incorrect email or password.

    const validPassword = await bcrypt.compare(password, user.passwordHash);
    if (!validPassword) return res.status(401).json({ error: "Incorrect email or

    const token = signToken(user);
    res.json({ token, user: { id: user.id, email: user.email, tier: user.tier } })
  } catch (error) {
    console.error("Login error:", error);
    res.status(500).json({ error: "Could not log in. Please try again." });
  }
});

```

```

app.get('/api/auth/me', requireAuth, async (req, res) => {
  try {
    const user = await prisma.user.findUnique({
      where: { id: req.auth.userId },
      select: { id: true, email: true, tier: true, masterKeyPurchased: true }
    });
    if (!user) return res.status(404).json({ error: "User not found." });
    res.json({ user });
  } catch (error) {
    console.error("Fetch current user error:", error);
    res.status(500).json({ error: "Could not load account." });
  }
});

```

```

// =====
// A. ACCESS CONTROL MIDDLEWARE
// =====

```

```

const enforceTier = (requiredTiers = []) => {
  return async (req, res, next) => {
    try {
      const userId = req.authUserId || req.headers['x-user-id'] || req.body.userId
      if (!userId) return res.status(401).json({ error: "Unauthenticated request."

      const dbUser = await prisma.user.findUnique({
        where: { id: userId },
        select: { tier: true }
      });

      if (!dbUser || !requiredTiers.includes(dbUser.tier)) {
        return res.status(403).json({
          error: `Access denied. Requires subscription status: [${requiredTiers.j
        });
      }

      req.userTier = dbUser.tier;
      next();
    } catch (error) {
      console.error("enforceTier error:", error);
      return res.status(500).json({ error: "Internal authorization check failed."
    }
  };
};

// =====
// B. AI ADVISOR ENGINE - tier-aware memory, no fabricated financial advice
// =====
const aiAdvisorService = {
  async getMemoryContextForUser(userId) {
    const user = await prisma.user.findUnique({ where: { id: userId }, select: {
    if (!user) return { recentMessages: [], profileSummary: null };

    if (user.tier === 'FREE') {
      const shortHistory = await prisma.chatMessage.findMany({
        where: { userId }, orderBy: { createdAt: 'desc' }, take: 3
      });
      return { recentMessages: shortHistory.reverse(), profileSummary: null };
    }

    const fullHistory = await prisma.chatMessage.findMany({
      where: { userId }, orderBy: { createdAt: 'desc' }, take: 50
    });
    const memoryProfile = await prisma.userMemoryProfile.findUnique({ where: { us

    return {

```

```

    recentMessages: fullHistory.reverse(),
    profileSummary: memoryProfile ? memoryProfile.summary : null
  };
},

async updateUserMemoryProfile(userId) {
  const user = await prisma.user.findUnique({ where: { id: userId }, select: {
    if (!user || user.tier === 'FREE') return;

    const messages = await prisma.chatMessage.findMany({
      where: { userId }, orderBy: { createdAt: 'desc' }, take: 30
    });

    // STUB: replace with a real summarization call (e.g. ask Claude to
    // summarize `messages` into 2-3 sentences) before shipping.
    const updatedDurableSummary = "Placeholder profile summary - wire up real sum

    await prisma.userMemoryProfile.upsert({
      where: { userId },
      update: { summary: updatedDurableSummary },
      create: { userId, summary: updatedDurableSummary }
    });
  },

  async processAdvisorMessage(userId, userPrompt) {
    const memory = await this.getMemoryContextForUser(userId);

    const systemPrompt = `
You are the AnalystIQ Advisor, a financial, business, investing, and accounting e
DEFAULT BEHAVIOR: Be thorough without waiting to be asked twice. When a user rais
TONE: Be direct and specific, not vague or hedgy. Use plain language over jargon;
MEMORY USAGE: You will be given either a short recent-message window only (FREE t
BOUNDARIES:
- Never give a direct personalized instruction to buy, sell, or invest a specific
- Never guarantee returns or outcomes for a business or investment.
- If a user appears to be making a high-stakes decision under financial distress,
- Do not fabricate specific figures, prices, or statistics not provided in contex
`.trim();

    await prisma.chatMessage.create({ data: { userId, role: 'user', content: user

    const contextBlock = memory.profileSummary
      ? `User profile summary: ${memory.profileSummary}\n\n`
      : '';
    const historyBlock = memory.recentMessages
      .map(m => `${m.role}: ${m.content}`).join('\n');

```

```

const msg = await anthropic.messages.create({
  model: "claude-sonnet-4-6",
  max_tokens: 1500,
  system: systemPrompt,
  messages: [
    { role: "user", content: `${contextBlock}Recent conversation:\n${historyB
  ]
});

const generatedText = msg.content[0].text;
await prisma.chatMessage.create({ data: { userId, role: 'assistant', content:
this.updateUserMemoryProfile(userId).catch(console.error);

return generatedText;
}
};

// =====
// C. FINANCIAL RESEARCH HANDLER - safety-checked
// =====
async function getLiveMarketData(ticker) {
  if (!ticker || !ENV.FINNHUB_STOCK_API_KEY) return null;
  try {
    const key = ENV.FINNHUB_STOCK_API_KEY;
    const quoteRes = await fetch(`https://finnhub.io/api/v1/quote?symbol=${ticker
    const quote = await quoteRes.json();
    const metricRes = await fetch(`https://finnhub.io/api/v1/stock/metric?symbol=
    const metrics = await metricRes.json();

    return `LIVE MARKET DATA (${ticker}):
- Current Price: ${quote.c}
- Today's Change: ${quote.dp}%
- 52-Week High: ${metrics.metric?.['52WeekHigh'] || 'N/A'}
- P/E Ratio: ${metrics.metric?.['peTTM'] || 'N/A'}
`;
  } catch (e) {
    console.error("getLiveMarketData error:", e);
    return " [Real-time data connection failed. Proceeding with caution.]";
  }
}

async function handleFinancialResearch(req, res) {
  const { userId, userPrompt, sectorContext, targetTicker } = req.body;
  try {
    const liveData = targetTicker ? await getLiveMarketData(targetTicker) : "";
    const verifiedKnowledge = await prisma.financialKnowledge.findFirst({
      where: { topic: sectorContext }

```

```

    });
    const bookContext = verifiedKnowledge ? verifiedKnowledge.content : "Standard

    const systemPrompt = `
You are a Financial Research Bot operating under a strict zero-hallucination cons
Your answers must be grounded completely in the provided Book Knowledge Reference
ANTI-FABRICATION & LEGAL BOUNDARIES:
1. Present balanced textbook arguments. Provide reasons to consider an asset and
2. If data is missing from the reference, state exactly what is missing rather th
3. If you must use a simulated or generic scenario because real data is missing,
    "[SIMULATED CONTEXT WARNING: The following analysis uses a generic, simulated
4. Be clear, direct, and specific. Define technical terms briefly when used.
`.trim();

    const msg = await anthropic.messages.create({
      model: "claude-sonnet-4-6",
      max_tokens: 2500,
      system: systemPrompt,
      messages: [
        { role: "user", content: `Live Data Context:\n${liveData}\n\nBook Rules:\n
      ]
    });

    const finalAnalysis = msg.content[0].text;

    await prisma.auditLog.create({
      data: {
        userId,
        prompt: userPrompt,
        response: finalAnalysis,
        status: finalAnalysis.includes('SIMULATED CONTEXT WARNING') ? 'SIMULATED_
      }
    });

    res.json({ success: true, analysis: finalAnalysis });
  } catch (error) {
    console.error("Financial research handler error:", error);
    res.status(500).json({ success: false, error: "System processing failure." })
  }
}

// =====
// D. MARKET DATA STUBS
// =====
const stockService = {
  async getMarketData(ticker) {
    if (!ENV.FINNHUB_STOCK_API_KEY) {

```

```

        return { symbol: ticker.toUpperCase(), currentPrice: 220.50, status: "Mock"
    }
    return await getLiveMarketData(ticker);
},

initializeRealTimeFeed(ioServerInstance) {
    if (!ENV.FINNHUB_STOCK_API_KEY) return;
    const socket = new WebSocket(`wss://ws.finnhub.io?token=${ENV.FINNHUB_STOCK_A

    socket.on('open', () => {
        socket.send(JSON.stringify({ type: 'subscribe', symbol: 'AAPL' }));
        socket.send(JSON.stringify({ type: 'subscribe', symbol: 'AMZN' }));
        socket.send(JSON.stringify({ type: 'subscribe', symbol: 'SPY' }));
    });

    socket.on('message', (data) => {
        const payload = JSON.parse(data);
        if (payload.type === 'trade') {
            payload.data.forEach(trade => {
                const humanReadableTime = new Date(trade.t).toLocaleTimeString([], { ho
                const normalizedTradeData = {
                    symbol: trade.s, price: trade.p, volume: trade.v, timeLabel: humanRea
                };
                if (ioServerInstance) ioServerInstance.emit('live-ticker-update', norma
            });
        }
    });
}

};

const metalService = {
    async getMetalsSpot() {
        // STUB: replace with a real metals price API call.
        return { gold: 2500.75, silver: 29.40, platinum: 960.10, ratio: 85.06, status
    }
};

// =====
// D2. MARKET OVERVIEW CACHE - GET /api/market/overview
// Free-tier market data APIs have strict rate limits, so this refreshes a
// tracked list on a timer instead of hitting the API on every page view.
// =====
const TRACKED_SYMBOLS = [
    'AAPL', 'MSFT', 'GOOGL', 'AMZN', 'NVDA', 'META', 'TSLA',
    'SPY', 'QQQ', 'DIA', 'IWM', 'VTI',
    'JPM', 'BAC', 'GS', 'V', 'MA',
    'WMT', 'COST', 'HD', 'MCD', 'NKE',

```

```

    'JNJ', 'UNH', 'PFE',
    'XOM', 'CVX',
    'BA', 'CAT',
    'DIS', 'NFLX', 'AMD', 'INTC'
  ];
  const MARKET_REFRESH_INTERVAL_MS = 60 * 1000;
  let marketCache = { updatedAt: null, quotes: [] };

  async function fetchQuoteForSymbol(symbol) {
    const apiKey = ENV.FINNHUB_STOCK_API_KEY;
    const quoteRes = await fetch(`https://finnhub.io/api/v1/quote?symbol=${symbol}&`
    const quote = await quoteRes.json();
    const profileRes = await fetch(`https://finnhub.io/api/v1/stock/metric?symbol=${symbol}&`
    const metrics = await profileRes.json();

    return {
      symbol,
      price: quote.c ?? null,
      change: quote.d ?? null,
      changePercent: quote.dp ?? null,
      high52w: metrics.metric?.['52WeekHigh'] ?? null,
      low52w: metrics.metric?.['52WeekLow'] ?? null,
      peRatio: metrics.metric?.peNormalizedAnnual ?? null,
      dividendYield: metrics.metric?.dividendYieldIndicatedAnnual ?? null,
      marketCap: metrics.metric?.marketCapitalization ?? null,
    };
  }

  async function refreshMarketCache() {
    if (!ENV.FINNHUB_STOCK_API_KEY) return; // don't burn a cycle with no key confi
    try {
      const results = await Promise.allSettled(TRACKED_SYMBOLS.map(fetchQuoteForSym
      const quotes = results.filter(r => r.status === 'fulfilled').map(r => r.value
      marketCache = { updatedAt: new Date().toISOString(), quotes };
      console.log(`Market cache refreshed: ${quotes.length}/${TRACKED_SYMBOLS.lengt
    } catch (error) {
      console.error("Market cache refresh failed:", error);
    }
  }

  refreshMarketCache();
  setInterval(refreshMarketCache, MARKET_REFRESH_INTERVAL_MS);

  app.get('/api/market/overview', (req, res) => {
    res.json({ updatedAt: marketCache.updatedAt, quotes: marketCache.quotes });
  });

  // =====

```

```

// D3. NEWS / FINANCIALS / EARNINGS
// =====
app.get('/api/market/news/:symbol', async (req, res) => {
  try {
    const symbol = req.params.symbol.toUpperCase();
    const apiKey = ENV.FINNHUB_STOCK_API_KEY;

    const to = new Date();
    const from = new Date();
    from.setDate(from.getDate() - 14);
    const fmt = (d) => d.toISOString().split('T')[0];

    const response = await fetch(
      `https://finnhub.io/api/v1/company-news?symbol=${symbol}&from=${fmt(from)}&`
    );
    const articles = await response.json();
    if (!Array.isArray(articles)) return res.json({ symbol, articles: [] });

    const cleaned = articles.slice(0, 20).map(a => ({
      headline: a.headline,
      summary: a.summary,
      source: a.source,
      url: a.url,
      image: a.image,
      datetime: a.datetime ? new Date(a.datetime * 1000).toISOString() : null
    }));
    res.json({ symbol, articles: cleaned });
  } catch (error) {
    console.error("News fetch error:", error);
    res.status(500).json({ error: "Could not load news right now." });
  }
});

app.get('/api/market/financials/:symbol', async (req, res) => {
  try {
    const symbol = req.params.symbol.toUpperCase();
    const apiKey = ENV.FINNHUB_STOCK_API_KEY;

    const response = await fetch(
      `https://finnhub.io/api/v1/stock/financials-reported?symbol=${symbol}&freq=`
    );
    const data = await response.json();
    if (!data.data || data.data.length === 0) return res.json({ symbol, statement

    const statements = data.data.slice(0, 3).map(filing => ({
      year: filing.year,
      filedDate: filing.filedDate,

```

```

        incomeStatement: filing.report?.ic || [],
        balanceSheet: filing.report?.bs || [],
        cashFlow: filing.report?.cf || []
    }));
    res.json({ symbol, statements });
} catch (error) {
    console.error("Financials fetch error:", error);
    res.status(500).json({ error: "Could not load financial statements right now."
    });
});

app.get('/api/market/earnings', async (req, res) => {
    try {
        const apiKey = ENV.FINNHUB_STOCK_API_KEY;
        const from = new Date();
        const to = new Date();
        to.setDate(to.getDate() + 14);
        const fmt = (d) => d.toISOString().split('T')[0];

        const response = await fetch(
            `https://finnhub.io/api/v1/calendar/earnings?from=${fmt(from)}&to=${fmt(to)}`
        );
        const data = await response.json();

        const events = (data.earningsCalendar || []).map(e => ({
            symbol: e.symbol,
            date: e.date,
            hour: e.hour,
            epsEstimate: e.epsEstimate,
            epsActual: e.epsActual,
            revenueEstimate: e.revenueEstimate,
            revenueActual: e.revenueActual
        }));
        res.json({ events });
    } catch (error) {
        console.error("Earnings calendar fetch error:", error);
        res.status(500).json({ error: "Could not load the earnings calendar right now"
        });
    }
});

// =====
// E. STRIPE SERVICE - three tiers + PDF add-on + verified webhook
// =====

const stripeService = {
    // --- FREE tier needs no Stripe call - new users default to FREE in your DB.

    // --- CORE tier: recurring subscription, monthly or annual ---

```

```

async createCoreSubscriptionCheckout(userId, billingCycle = 'monthly') {
  const priceId = billingCycle === 'annual'
    ? ENV.STRIPE_PRICE_CORE_ANNUAL
    : ENV.STRIPE_PRICE_CORE_MONTHLY;

  if (!priceId) throw new Error(`Missing Stripe price ID for Core ${billingCycl

const session = await stripe.checkout.sessions.create({
  mode: 'subscription',
  payment_method_types: ['card'],
  line_items: [{ price: priceId, quantity: 1 }],
  allow_promotion_codes: true,
  client_reference_id: userId, // lets the webhook know which user paid
  success_url: 'https://yoursite.com/success?tier=core',
  cancel_url: 'https://yoursite.com/cancel',
});
return session.url;
},

// --- MASTER KEY: one-time payment, lifetime access + PDF guide ---
async createMasterKeyCheckout(userId) {
  if (!ENV.STRIPE_PRICE_MASTER_KEY) throw new Error("Missing Stripe price ID fo

const session = await stripe.checkout.sessions.create({
  mode: 'payment',
  payment_method_types: ['card'],
  line_items: [{ price: ENV.STRIPE_PRICE_MASTER_KEY, quantity: 1 }],
  allow_promotion_codes: true,
  client_reference_id: userId,
  metadata: { product: 'master_key' }, // used by the webhook instead of gues
  success_url: 'https://yoursite.com/success?tier=master',
  cancel_url: 'https://yoursite.com/cancel',
});
return session.url;
},

// --- Standalone PDF-only purchase ---
async createPDFCheckout(userId) {
  if (!ENV.STRIPE_PRICE_EBOOK) throw new Error("Missing Stripe price ID for the

const session = await stripe.checkout.sessions.create({
  mode: 'payment',
  payment_method_types: ['card'],
  line_items: [{ price: ENV.STRIPE_PRICE_EBOOK, quantity: 1 }],
  allow_promotion_codes: true,
  client_reference_id: userId,
  metadata: { product: 'ebook' },

```

```

        success_url: 'https://yoursite.com/success?item=pdf',
        cancel_url: 'https://yoursite.com/cancel',
    });
    return session.url;
},

// --- Webhook: verifies Stripe's signature for real, then updates the user's t
async handleWebhook(rawBody, sig) {
    let event;
    try {
        event = stripe.webhooks.constructEvent(rawBody, sig, ENV.STRIPE_WEBHOOK_SEC
    } catch (err) {
        // Signature didn't match - this request did NOT come from Stripe.
        console.error("Webhook signature verification failed:", err.message);
        throw { status: 400, message: `Webhook Error: ${err.message}` };
    }

    if (event.type === 'checkout.session.completed') {
        const session = event.data.object;
        const userId = session.client_reference_id;
        if (!userId) return { received: true, note: "No client_reference_id on sess

        if (session.mode === 'subscription') {
            await prisma.user.update({
                where: { id: userId },
                data: {
                    tier: 'CORE',
                    stripeCustomerId: session.customer,
                    stripeSubscriptionId: session.subscription,
                }
            });
        } else if (session.mode === 'payment') {
            // Uses metadata (set above) rather than guessing from amount_total -
            // more robust if prices ever change.
            if (session.metadata?.product === 'master_key') {
                await prisma.user.update({
                    where: { id: userId },
                    data: { tier: 'MASTER', masterKeyPurchased: true, stripeCustomerId: s
                });
            }
            // ebook purchases don't change tier - just record the purchase
            // elsewhere if you want to track ebook-only buyers.
        }
    }

    if (event.type === 'customer.subscription.deleted') {
        const subscription = event.data.object;

```

```

        const user = await prisma.user.findFirst({ where: { stripeSubscriptionId: s
        if (user) {
            await prisma.user.update({ where: { id: user.id }, data: { tier: 'FREE' }
        }
    }

    return { received: true };
}
};

// =====
// F. REST API ENDPOINTS
// =====
app.use(express.json()); // JSON body parsing for all routes EXCEPT the webhook b

app.post('/api/finance/research', async (req, res) => {
    await handleFinancialResearch(req, res);
});

app.get('/api/market/stocks/:ticker', async (req, res) => {
    const data = await stockService.getMarketData(req.params.ticker);
    res.json(data);
});

app.get('/api/market/metals', async (req, res) => {
    const data = await metalService.getMetalsSpot();
    res.json(data);
});

app.post('/api/advisor/chat', requireAuth, async (req, res) => {
    const { message } = req.body;
    const reply = await aiAdvisorService.processAdvisorMessage(req.authUserId, mess
    res.json({ response: reply });
});

app.post('/api/finance/business', enforceTier(['CORE', 'MASTER']), async (req, re
    res.json({ success: true, info: "Premium-gated enterprise matrix unlocked." });
});

// --- Checkout endpoints, one per tier/product ---
app.post('/api/checkout/core', requireAuth, async (req, res) => {
    try {
        const { billingCycle } = req.body; // billingCycle: 'monthly' | 'annual'
        const url = await stripeService.createCoreSubscriptionCheckout(req.authUserId
        res.json({ url });
    } catch (error) {
        console.error(error);
    }
}

```

```

        res.status(500).json({ error: "Checkout session failed to initialize." });
    }
});

app.post('/api/checkout/master', requireAuth, async (req, res) => {
    try {
        const url = await stripeService.createMasterKeyCheckout(req.authUserId);
        res.json({ url });
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Checkout session failed to initialize." });
    }
});

app.post('/api/checkout/pdf', requireAuth, async (req, res) => {
    try {
        const url = await stripeService.createPDFCheckout(req.authUserId);
        res.json({ url });
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Checkout session failed to initialize." });
    }
});

// --- Stripe webhook - MUST use express.raw(), not express.json(), or
// signature verification will fail. Registered with its own raw parser here,
// so it needs to be defined before/independent of the app.use(express.json()) ab
app.post('/api/webhooks/stripe', express.raw({ type: 'application/json' }), async
    try {
        const result = await stripeService.handleWebhook(req.body, req.headers['strip
        res.json(result);
    } catch (err) {
        res.status(err.status || 500).send(err.message || "Webhook processing failed.
    }
});

module.exports = app;

```